DESIGN DOCUMENT - LAB 3

Shireen Meher
shireenmeher@umass.edu

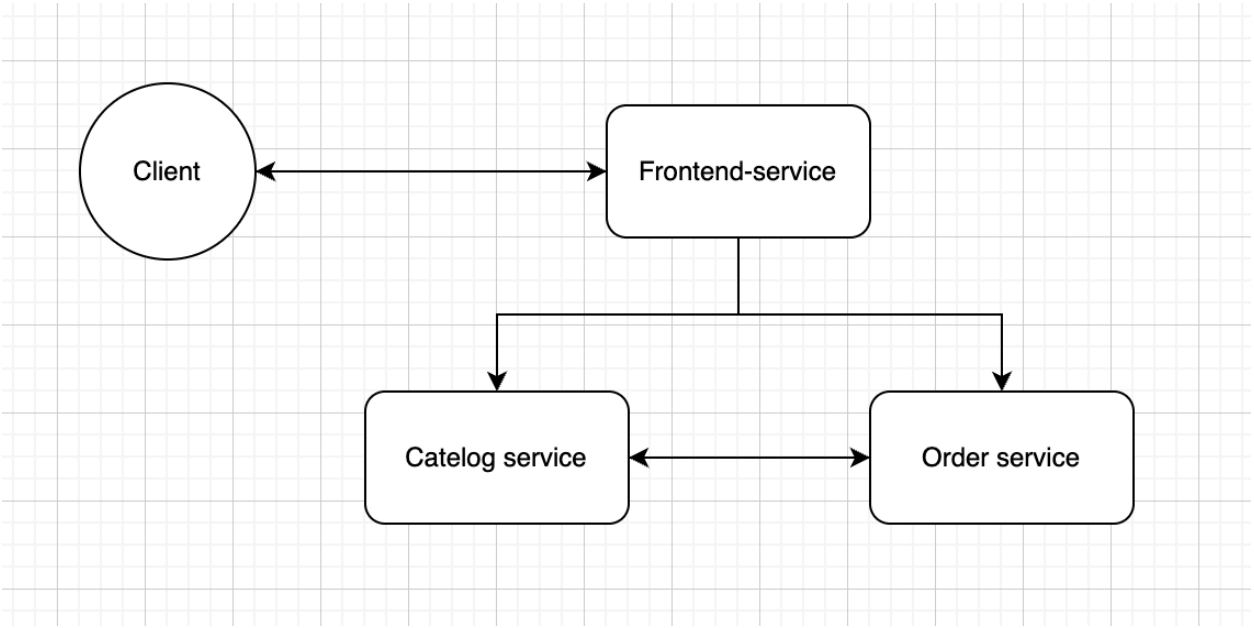
Overview

This system simulates a basic stock trading platform composed of three microservices:

1. Frontend Service
2. Order Service
3. Catalog Service

Each service communicates via HTTP, supports concurrent requests, and persists critical data to disk. The services are containerized using Docker and orchestrated using Docker Compose.

Microservice architecture



Folder Structure

```
project-root/
├── docker-compose.yml
├── data/
│   ├── orders.csv            # Order history
```

```
│   └── catalog.csv            # Catalog state
├── frontend_service/
│   ├── main.py                # Frontend HTTP gateway
│   └── test_main.py           # Unit tests
├── order_service/
│   ├── main.py                # Order placement logic
│   └── test_main.py           # Unit tests
├── catelog_service/
│   ├── main.py                # Catalog inventory logic
│   └── test_main.py           # Unit tests
├── functional/
│   └── test_lookup_validation.py  # Functional test scripts (latency)
│   └── test_trade_validation.py  # Functional test scripts (latency)
│
├── logs/
│   ├── lookup_latency_nondocker_1_run1.log
│   └── trade_latency_docker_5_run3.log
├── Dockerfile
└── .env                       # Environment variable configuration
```

Frontend Service

- Acts as the **gateway** for client interactions.
- Handles:
  - `GET /lookup/{stock_name}` → For checking stock price/quantity.
  - `POST /trade` → To trade buy/sell stock.

- Makes HTTP calls to:
  - Catalog Service (for lookup)
  - Order Service (for trade)

Concurrency:

- Uses `ThreadPoolExecutor` to handle multiple clients concurrently.
- Max workers = 5

Unit test cases:

- `test_main.py` with mock requests and dummy HTTP handlers.

- Libraries used - unittest

HTTP Server:

- Built using `http.server.HTTPServer` + `BaseHTTPRequestHandler`
- Concurrent clients are supported via `ThreadPoolExecutor(max_workers=5)`
- Used thread-per-session model
- Reference: https://www.codementor.io/@joaojonesventura/building-a-basic-http-server-from-scratch-in-python-1cedkg0842

Catalog Service

- Manages the **stock catalog** (price, quantity).
- Supports:
  - `GET /lookup/{stock_name}`
  - `POST /update/{stock_name}/{change}` → Increment/decrement quantity

Data Storage:

- In-memory `catalog` dictionary
- Persisted periodically to `data/orders.csv` using a background thread every 10 secs

Synchronization:

- Uses threading.Lock to protect shared catalog data

Concurrency:

- Multithreaded using `ThreadPoolExecutor`
- Max_workers = 5

HTTP Server:

- Built using http.server.HTTPServer + BaseHTTPRequestHandler
- Concurrent clients are supported via ThreadPoolExecutor(max_workers=5)

- Used thread-per-session model

Order Service

Responsibilities:

- Validates and processes buy/sell orders
- Records orders in in-memory orders_data after successful transaction with an incremental transaction_id
- Updates catalog service stock via HTTP

Data Storage:

- Transaction log saved to `data/orders.csv`
- Transaction ID is auto-incremented and protected with a lock

HTTP Server:

- Built using `http.server.HTTPServer` + `BaseHTTPRequestHandler`
- Concurrent clients are supported via `ThreadPoolExecutor(max_workers=5)`
- Used thread-per-session model

Flow:

1. Receives order → validates
2. Calls catalog service to update stock
3. If success → records order and responds with transaction ID

Deployment & Orchestration

- Each service has its own Dockerfile AND .env files to store environment variables
- Uses venv and .env files for config

Data Persistence:

- ./data folders mapped to containers
- Ensures stock catalog and order logs survive container restarts

API Contract

1. Lookup Request

Endpoint:
- GET /lookup/{stock_name}
- Retrieves the price and available quantity of a specific stock.

Request:

- **Method:** GET
- **URL Example:** `http://<frontend_host>:<port>/lookup/GameStart`

Response:

Success:

```
{200, "data": {
"Name": stcok_name,
  "price": 15.99,
  "quantity": 100
}}
```

Error:

```
{
  "error": "Invalid JSON or fields"
}
```

2. Trade Request:

Endpoint:
- POST /trade
- Initiates a buy or sell transaction for a stock.

Request:

- **Method:** POST
- **Sample URL:** `http://<frontend_host>:<port>/trade`
- **Content-Type:** application/json

Request Body:

```
{
  "name": "GameStart",
  "type": "buy",
  "quantity": 2
}
```

Successful Response:

- **Status:** 200 OK
- **Content-Type:** application/json

```
{
  "data": {
    "transaction_number": 5
  }
}
```

Validation Error Response:

- **Status:** 400 Bad Request

```
{
  "error": "Invalid JSON or fields"
}
```

Testing Strategy

Unit Testing

- Each service has test_main.py
- Uses unittest, patch, and MagicMock
- Covers:
    - Success paths (valid trades/lookups)
    - Error scenarios (bad inputs, service failures)

Functional Testing

- Simulated concurrent clients (1–5)
- Logged average latency for:
    - Lookup (read-only)
    - Trade (write + multi-service call)
- Plotted latency vs concurrency graphs

Load testing

- Have written a client.py file that has code for:
    - N sequential trade requests
    - N sequential lookup requests
    - N sequential lookup requests with trade requests of a probability p
- Each such client.py is deployed concurrently from 1 to 5 numbers to perform the load testing

---

Design for Replicated Order Service

1. Overview

- This application implements a replicated order service system for a stock trading platform. The system supports:
    - Leader election
    - Replication of order data
    - Fault tolerance for crash failures

- Client-facing frontend that routes requests to the leader

2. Architecture

- Order Service (3 replicas)
- Catalog Service
- Frontend Service (acts as proxy to leader)
- Dockerized setup with docker-compose
- Persistent volume for each order replica

3. Components

- Order Service:
  - Handles order creation and queries.
  - Maintains local persistent state (orders.json).
  - On startup, performs sync with other replicas if needed.
  - Exposes:
    - POST `/orders` – Add new order (only by leader)
    - GET `/orders/<order_id>` – Get order info
    - GET `/orders/sync/<last_order_id>` – For state syncing
    - POST `/set_leader` – Mark this replica as leader

- Catalog Service:
  - Provides static catalog data for clients (e.g., stock list).

- Frontend Service:
  - Handles user requests
  - Detects current leader (via GET `/leader`)
  - Routes all POST/GET requests to the leader
  - Re-elects leader if unresponsive

4. Leader Election

- Highest order_service ID (1 > 2 > 3) becomes leader.
- Frontend initiates leader election if timeout on request.
- Each replica exposes a health endpoint `/ping`.

5. Failure Handling

- When a replica crashes and restarts:
  - It contacts other replicas to sync missing order data.
  - Clients transparently interact with the leader only.

    - Inconsistent replicas are detected using order logs.

6. Data Consistency

- Strong consistency assumed via leader-based writes.
- Followers sync missed orders on recovery.
- Backend storage is simple file-based JSON.

7. Future Enhancements

- Paxos/Raft for consensus.
- Secure auth layer.
- Real database support (e.g., PostgreSQL).

Performance Considerations:

- **Latency**: Measured response time under **1 to 5 concurrent clients**.
- **Throughput**: Number of **requests per second** processed by the server.